

- The control block (contains meta data)
- The object being managed

```
#include <iostream>
#include <memory>

using namespace std;
class smartclass {
    string name;
public:
    smartclass(string str)
    {
        name = str;
    }

    int display()
    {
        cout << name<<endl;
    }
};

int main()
{
    shared_ptr<smartclass> P1(new
smartclass("smartclass"));
    cout << "P1:";
    P1->display();

    shared_ptr<smartclass> P2;
    P2 = P1;

    cout << "P2:";
    P2->display();
    cout << "reference count
:"<<P1.use_count() << endl;

    shared_ptr<smartclass> P3(new
smartclass("myclass"));
    cout << "P3 :";
    P3->display();
    cout << "is p1 unique ? " <<
P1.unique()<<endl;
    cout << "is p3 unique ? " <<
P3.unique()<<endl;

    P3.swap(P2); // swapping of
pointers
    cout << "P3 after swapping :";
    P3->display();
```

```
        return 0;
    }
```

The output is:

```
P1 :smartclass
P2 :smartclass
reference count :2
P3 :myclass
is p1 unique ? 0
is p3 unique ? 1
P3 after swapping :smartclass
```

In the above program, P1 and P2 point to the same resource; hence, the reference count yields 2.

Then we check whether the pointer is unique or not, to find out whether the ownership is shared. For P1, ownership is shared, so *P1.unique()* returns 0. For P3, *P3.unique()* returns 1, which means its ownership is not shared with any other pointer. Finally, we swap P3 with P2 just to show that *shared_ptr* supports pointer swapping.

weak_ptr: Using *weak_ptr*, it is possible to access the object but not own it. To use *weak_ptr*, the object must be already owned by some other pointer (*shared_ptr*). For example, if a shared pointer *obj1* owns a resource, the same resource can be accessed by another *weak_ptr* without owning it.

- Used to avoid cyclic reference between *shared_ptr* references.
- It must be converted to *shared_ptr* in order to access the referenced object.

```
#include <iostream>
#include <memory>

using namespace std;
int main()
{
    weak_ptr<int> wptr;
    shared_ptr<int> sp = std::make_
shared<int>(10);
    wptr = sp;

    cout << "use_count = :"<<wptr.
```

```
use_count();
    shared_ptr<int> sptr = wptr.
lock();
    cout <<"\nvalue
:"<<*sptr<<endl;
}
```

The output is:

```
use_count = :1
value :10
```

In the above program, change the *weak_ptr* to *shared_ptr* and comment the last two lines of code; then compile and run, and the *use_count* value yields 2. On the contrary, if we use exactly the same code as above, the *use_count* won't increase as the *weak_ptr* doesn't take ownership.

- *weak_ptr::lock()*: Creates a *shared_ptr* to manage the object.
- *weak_ptr::use_count()*: Returns the number of *shared_ptr* objects.
- *Std::make_shared*: Makes a single heap-allocation for both the control block and the managed object.

The aim of this article is to make each and every reader understand smart pointers. In today's world, everything is driven by software, directly or indirectly. But to make the software reliable and stable, it is imperative to understand concepts like smart pointers; otherwise, one has to spend many man-hours debugging and fixing issues. During my career, I have fixed many pointer related issues and noticed that many programmers don't have the patience to write code for releasing the resource. Smart pointers are very helpful in minimising bugs. Finally, coding is easy if we understand the purpose and usage of different methods. **END** 🐧

By: Sree Venkatesh

The author is an engineer by profession, with 6+ years of experience in the field of embedded Linux development.